

Turitin style system (Group One)

Software Requirements Specification

Aidan Wood, Lucas Pate, Troy Neal, Sarah Howcroft, Kevin Gracey and Steve Mastromarino

April 22, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Intended Audience	3
1.3	Intended Use	3
1.4	Product Scope	3
2	Overall Description	4
2.1	Product perspective	4
2.2	User Classes	4
2.3	Operating Environment	4
2.4	Constraints	4
2.5	Assumptions and Dependencies	5
3	System Features and Requirements	5
3.1	System Features	5
4	Data requirements	5
4.1	Logical data model	5
4.2	Data dictionary	6
4.3	Reports	6
4.4	Data acquisition, integrity and disposal	6

5	External interface requirements	7
5.1	User Interfaces	7
5.1.1	User Roles and Access	7
5.1.2	User Interface Characteristics	7
5.1.3	Common User Interface Elements	7
5.1.4	User Interface Privacy Requirements	8
5.1.5	Error reporting and Validation	8
5.1.6	Accessibility and Usability	8
5.1.7	Interface Requirements for User Interface Components	9
6	Quality Attributes	9
6.1	Usability	9
6.2	Performance	10
6.3	Security	10
6.4	Safety	11
7	Internalization and localization requirements	11
8	Other requirements	11
8.1	Maintainability	11
8.2	Data Loss	12
8.3	Compatibility	12
9	Analysis models	12
9.1	Data Dictionary	12
9.2	State Transition Diagram	13
9.3	Feature Tree	14
10	Diagrams	14

1 Introduction

1.1 Purpose

This document exists to describe the features and use cases of our Turnitin style program. This document will feature both functional and non functional requirements of the system, use cases of the system and any safety mechanisms that are required

1.2 Intended Audience

Document is to be read by all who are involved in the process of creating the system as well as our client. Document is to be shown to our client before and major system work is completed.

1.3 Intended Use

The system should accurately and neatly compare different student submissions and search for and identify cases of plagiarism. Use cases include

1. Professor uses system to check student submissions
2. Student uses system to Submit their work to classes
3. Professor uses system to easily manage student submissions

1.4 Product Scope

The system is comprised of three major components; The engine for comparison, the repository to hold user submissions, and the website that the user interacts with to utilize the engine. The engine should effectively compare different submissions within the repository, the engine should support C, C++ and java for submissions. The website should provide professors with the ability to see the percent of plagiarism within a given paper and the size of each plagiarized section, as well as being shown an index of instances of potential plagiarism. Any personnel details of students should be removed or hidden by the system on submission.

2 Overall Description

2.1 Product perspective

Program will serve as a competitor or replacement to popular academic tool "Turnitin". Program aims to achieve greater accuracy than the current product standard.

2.2 User Classes

Users Will mainly fit into the two following user classes.

- Professor / Instructor / Teacher : This user should be able to view groups of entry's of the second group of user, being students. This user should be able to directly compare differing submissions of their choosing, however they should not have access to students outside their class, they are not an admin, they are more akin to group leaders
- Student / Guest : This "user" is not actually found within the database, that is to say Students are anonymous, and cannot sign in to the site. Students have the ability to make submission to a class using an assignment code they are given. Students will provide their name and ID during submission

2.3 Operating Environment

Program will be hosted on a web server, thus any device capable of accessing the web should interact properly with the system. Mainly, windows, mac and mobile operating systems should all function as intended.

Program UI will only accommodate english users, but there is nothing stopping those who speak other languages from using the system.

Program will be hosted on Railway.com's cloud servers, thus the site and repository will be hosted on American servers.

2.4 Constraints

- We will be making our own tokenizer, assuring this works properly is now on our shoulders
- We will be using Railway for our website, and supabase for our database. We have a premium subscription to railway so there should be no issues there, but our free plan of supabase does have a limited storage per item, thou that likely wont be an issue

- Functionality of the engine will be limited to what is possible in java, since that is our chosen language.

2.5 Assumptions and Dependencies

1. At some point there will be an issue in connecting the engine, repository and website. Time should be allocated to this possibility
- 2.
3. **Railway and supabase will be sufficient for both our site and repository**

3 System Features and Requirements

3.1 System Features

1. The main features of the system will be evaluating code files to test whether they are new or copied from previous submitted code. If this was used for real assignment submissions the professor can see whether submitted code has been copied from elsewhere and how much of it is new from the specific user. It will help to test for plagiarism and determine what percentage of the code was actually written by the submitting user. The repository system will support C, C++ and Java. Assignments will be able to be stored in the repository so that **new submissions will be compared with all submissions also found within that assignment** There will be a comparison engine to compare the submissions. The professor has the ability to view analysis of submissions and compare them in a web-based environment.

4 Data requirements

4.1 Logical data model

.

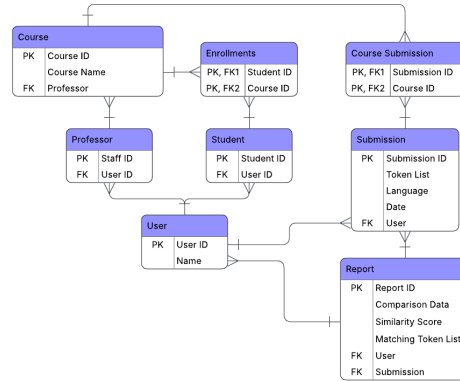


Figure 1: A logical data model showing relations between users, courses, and submissions.

Item	Type	Description
Report ID	int	Primary Key
Comparison Data	char(1)	Global, Course-wide, or Individual
Similarity Score	int	Percentage of Similar Content
Matching Token List	Token[]	List of Similar Tokens
User ID	int	Key of User who Created the Report
Submission ID	int	Key of Submission

Figure 2: A data dictionary describing the fields of the report class.

4.2 Data dictionary

4.3 Reports

Reports should only be able to be generated by the professor, students should never be able to create reports, or view their plagiarism percent in any way. Reports will highlight any instances of plagiarism and each index can be clicked to show the corresponding file it was stolen from. Professors can also export repository's (meaning all submissions found in an assignment) to a txt document.

4.4 Data acquisition, integrity and disposal

All submissions will be stored in the submission database; however, the submission database and all user information will be encrypted. The name of any user who submits their code will only be accessible to the course professor, if applicable. Reports will not display the information of other submissions that are flagged for similarity. **Student names and ID can only be found on comparison reports, they are not shown anywhere else**

5 External interface requirements

5.1 User Interfaces

This section describes the logical characteristics of the user interfaces provided by the system, including user roles, interface behavior, common elements, privacy considerations, and accessibility requirements.

5.1.1 User Roles and Access

The system will provide a web-based interface for the users. These users will be split into two main roles each with their own respective interactions:

- Instructor
 - Create an assignment with due dates and the ability to accept submissions.
 - Create a similarity report for all submissions in an assignment.
 - Create "boiler plate code" to be excluded from plagiarism detection.
 - Export all submissions in an assignment to a txt file.
 - Review analysis results.
- Student
 - Make submissions to an open assignment.

5.1.2 User Interface Characteristics

The user interface will be implemented as a web application primarily servicing desktop users with optional basic support for mobile users. Desktop users will be prioritized due to the high text density when comparing reports and viewing submissions as well as the expected environment they will be accessing the application from (Work/School).

5.1.3 Common User Interface Elements

- All Pages
 - Navigation Bar
 - Account Side Bar
- Occasional Reuse

- Buttons for: Submit, Upload, **Sign in, Sign out**
- Feedback Toasts
 - Success Messages
 - Error Messages
 - Warning Messages
- Progress Indicators

5.1.4 User Interface Privacy Requirements

Confidentiality is a main priority especially when some instructors may allow their submission repositories to be used / compared against. At no time should an instructor be able to view the personal information of a student that **has not made a submission to a class they have ownership over**. Additionally, students should not be able to access other students assignments or information, **or access their own information**

5.1.5 Error reporting and Validation

Validation should occur on both the client-side, and the server-side with appropriate levels of technical jargon, verbosity, and information. On the client-side error messages should be:

- Actionable and clear for example "Upload be one of the following types: .c, .cpp, .java".
- Non-technical, with no stack traces, unclear error codes, or back end specific jargon.
- Privacy first with no references to internal file paths, other users, or any other private information.
- **More specific information on errors can be found on our web hosting service, railway. This information will not be available to users but will be helpful for debugging**

5.1.6 Accessibility and Usability

The user interface should conform to industry and government standards for accessibility, implementing all of but not limited to the following:

- Keyboard navigable components, forms, and buttons.
- Appropriate colour contrast.
- Readable font sizes.
- Searchable tables with appropriate alt text.

5.1.7 Interface Requirements for User Interface Components

The following components will require some form of back end interaction or API:

- User / Session authentication
- File submission
- Assignment creation and tracking
- Repository access
- Report creation and viewer

6 Quality Attributes

6.1 Usability

REQ 1 - Smart UI Design:

- UI should be designed with thought and should highlight the most important features of the product: Logging in and creating accounts, **the ability to submit to assignments based on a given course code, and professors should be able to easily compare and view submissions**

REQ 2 - Dashboards

- Using a central dashboard for student use to easily go to and from their classes and their assignments to submit or check their scores Use of a central board for instructors to use to easily manage their courses

REQ 3 - Compatibility

- This system should work across a large range of web browsers and devices.

REQ 4 - Error Readability

- This system should give understandable error messages when errors happen like invalid file formats etc.

6.2 Performance

REQ - 1 Timing

- Submitted Assignments should give a confirmation within 5 seconds of user submission (not factoring the users internet speed)
- Response Times for all requests should take no longer than a second

REQ - 2 Multiple Users

- System should be able to be used by multiple users at once without noticeable slow-down.

REQ - 3 Responsive

- Website should not feel slow to navigate.

REQ - 4 Accuracy

- The system should accurately judge plagiarism within an acceptable range.

6.3 Security

REQ - 1 Account Security

- **Professors** will need to authenticate themselves to access data from their account.
- Accounts need to be secure from tampering from external sources and need to safeguard account related data to only the authorized users.
- Need to keep the database data from being seen by unauthorized users.

REQ - 2 Encryption

- System should encrypt data sent from User to Server and vice versa

REQ - 3 Leaking

- Through use of the plagiarism similarity function, users should not be able to forge solutions of other students.

REQ - 4 Logging

- Should keep a log of information of all login attempts.

6.4 Safety

REQ - 1 Information Hiding

- Both User data, and as much inner workings of the system as possible should be obfuscated from the front end user.

REQ - 2 Judgment

- Punishments should not be automatically given out for plagiarism penalties

REQ - 3 Challenges

- Students should be able to challenge their scores for review by the teaching staff
- A copy of the original code will need to be kept and saved.

REQ - 4 Data Storage

- How and what data is stored from Users should be easily available and readable.

7 Internalization and localization requirements

The internalization and localization requirements ensure that the system will be able to handle different programming languages such as C, C++ and Java. The system should be able to handle different coding styles and conventions so false positives don't get triggered when the code isn't copied or reused. It needs to be able to detect plagiarism across different languages such as if someone translated Java code for a program into C code for the same program. Date, time and number formats in produced reports must translate to the users local timezone. The UI needs to be flexible to support text expansions, right to left scripts and different commenting languages. The repository should be able to handle submission time stamps in the users local time zone.

8 Other requirements

8.1 Maintainability

REQ 1 - Modularity:

- Code should be designed to be easily maintainable and for sections/functions of the code to be swapped and changed with minimal collateral effects

REQ 2 - Automated Testing

- All systems should support use of automated testing

8.2 Data Loss

REQ 1 - Minimal Loss From Failure

- Should some error occur (crash, power failure, malicious attack etc.) The server should lose as little data as possible, optimally none.

8.3 Compatibility

REQ 1 - Code Compatibility

- The system should accept projects coded in C, C++ and Java

REQ 2 - Zips

- System should be able to receive projects in .zip format

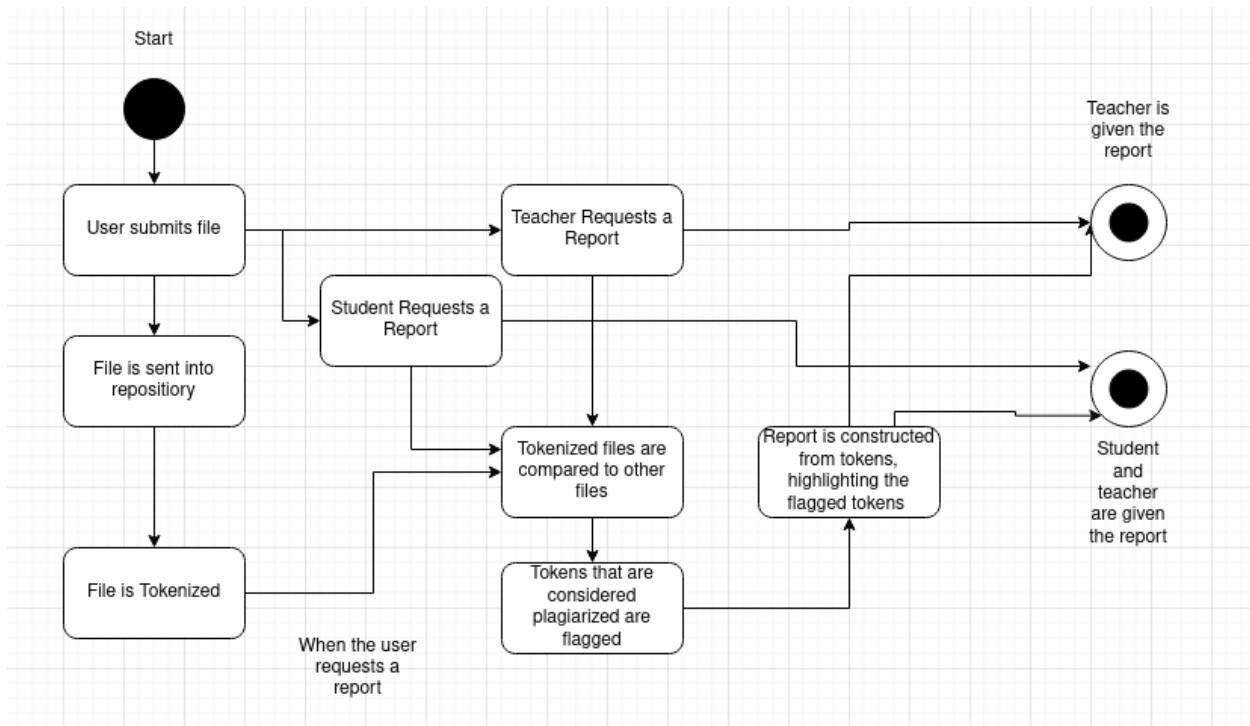
9 Analysis models

9.1 Data Dictionary

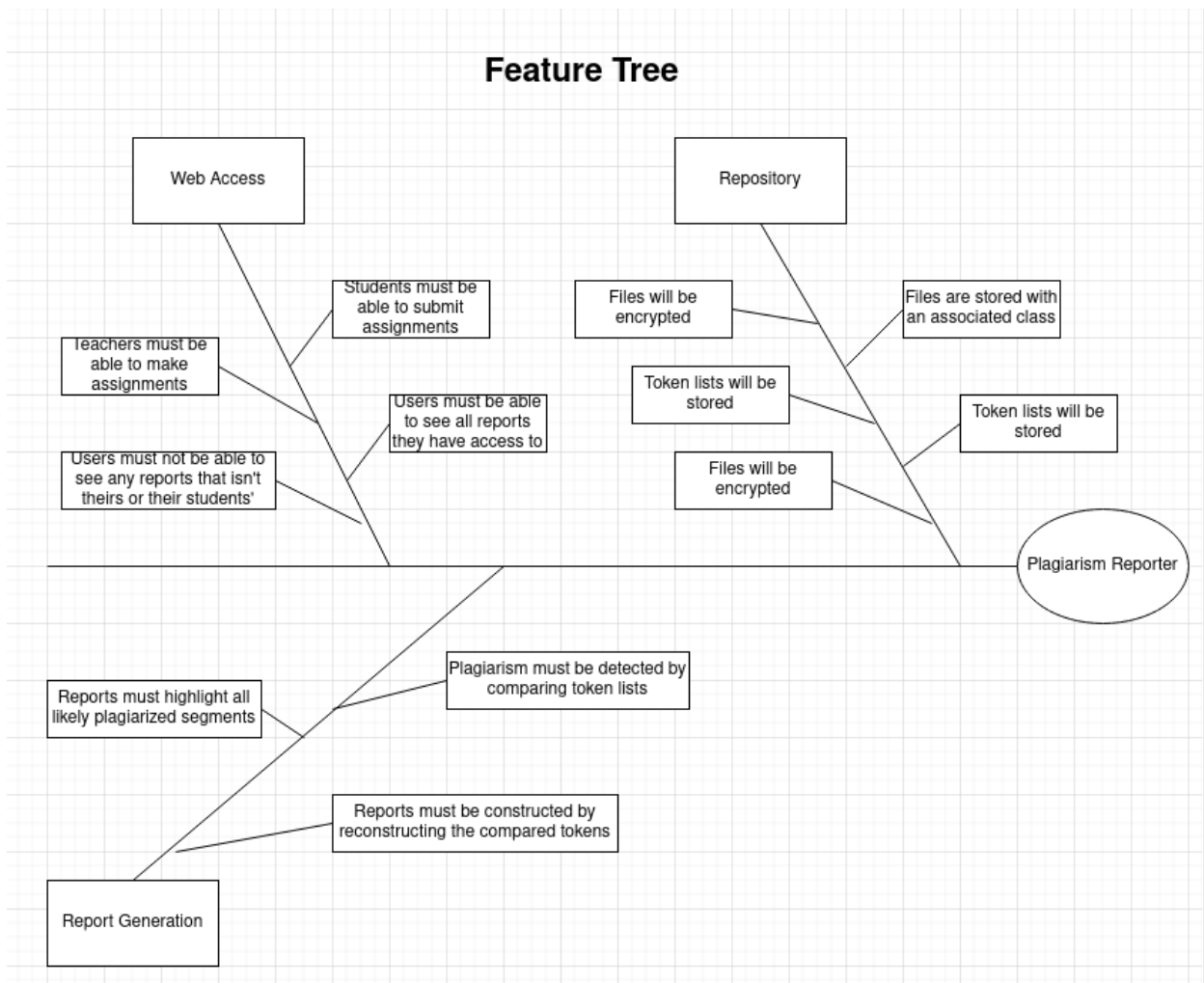
Class objects store the class list, and assignments lists for the class. User objects store the user's type and permissions across multiple classes. Students Assignment objects have a class, an owner, and a list of submission objects.

Students input Java, C, and Python files. Report Objects have an owner, an instructor (these may be the same user,) a score, and a token list. Submission objects store the submitted user files, and the user's personal information encrypted.

9.2 State Transition Diagram



9.3 Feature Tree



10 Diagrams

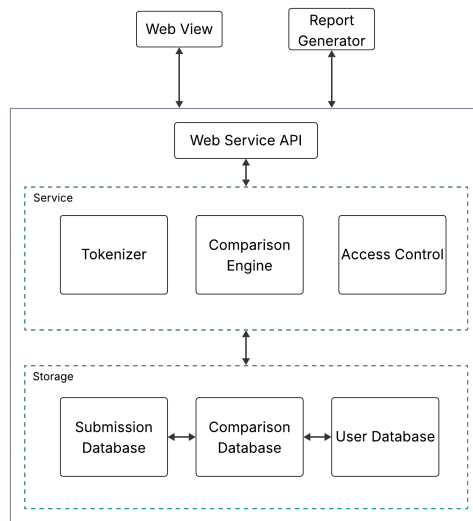


Figure 3: An architecture diagram showing the service and storage layers within the repository.